Revision Games — 2.3 Algorithms

OCR A Level Computer Science H446, Component 02, Section 2.3.1 — Algorithms.

Print, cut out, and play. All complexities verified against the OCR H446 spec.

Loop cards (dominoes)

How to play: Print and cut into 16 cards. Deal them all out. One player reads the **"Who has…"** question at the bottom of their card. The player holding the matching **"I have…"** answer reads it aloud, then reads their own question. Continue until you return to the starting card — the loop is closed. Time the class and try to beat your record. (You can verify the loop by following the chain below: it visits all 16 cards exactly once and returns to card 1.)

#	I have…	Who has…
1	**Merge sort**	the **worst-case** time complexity of binary search?
2	**O(log n)**	the data structure that is **LIFO**?
3	**Stack**	the traversal that outputs a BST in **ascending order**?
4	**In-order traversal**	the **average-case** complexity of quick sort?
5	**O(n log n)**	the search that requires the list to be **sorted**?
6	**Binary search**	the **worst-case** complexity of quick sort?
7	**O(n²)**	the data structure that is **FIFO**?
8	**Queue**	the path-finding algorithm that uses a **heuristic**?
9	**A* search**	the complexity of pushing onto a stack?
10	**O(1)**	the sort that picks a **pivot** and partitions the list?
11	**Quick sort**	the **best-case** complexity of insertion sort?
12	**O(n)**	the traversal order **Left → Right → Node**?
13	**Post-order traversal**	the sort that repeatedly swaps **adjacent** elements?
14	**Bubble sort**	the algorithm that finds the shortest path to **all** nodes with no heuristic?
15	**Dijkstra's algorithm**	the sort that recursively **splits then merges** sublists?
16	**Merge sort** (answer to 15) → loops back	the **worst-case** time complexity of binary search? *(= card 1's question)*

Verified loop chain (16 links, closes): 1 (Merge sort) asks binary search worst case → **O(log n)** = card 2 → asks LIFO → **Stack** = card 3 → asks ascending BST traversal → **In-order** = card 4 → asks quick sort average → **O(n log n)** = card 5 → asks search needing sorted list → **Binary search** = card 6 → asks quick sort worst → **O(n²)** = card 7 → asks FIFO → **Queue** = card 8 → asks heuristic path-finder → **A*** = card 9 → asks stack push cost → **O(1)** = card 10 → asks pivot/partition sort → **Quick sort** = card 11 → asks insertion sort best → **O(n)** = card 12 → asks Left→Right→Node → **Post-order** = card 13 → asks adjacent-swap sort → **Bubble sort** = card 14 → asks shortest-path-to-all, no heuristic → **Dijkstra** = card

15 → asks split-then-merge sort → **Merge sort** = card 16/1 → asks binary search worst case → back to card 2. **Loop closed.**

> Note: cards 1 and 16 both answer "Merge sort". Printed as 16 physical cards, card 16's "Who has…" matches card 2's "I have…", and card 1 closes the ring. If you prefer exactly one Merge sort card, use 15 cards and let card 15 (Dijkstra) ask the merge-sort question, with card 1 (Merge sort) answering it — also a valid closed loop.

Taboo cards

How to play: In pairs/teams. One player describes the **TERM** at the top without saying any of the 4 **BANNED** words (or obvious variants). Teammates guess. 1 point per correct guess in 60 seconds; pass allowed. Swap describer each round.

#	TERM	Banned words
1	**Binary search**	sorted, half, middle, log
2	**Merge sort**	divide, split, merge, recursive
3	**Big O notation**	complexity, time, growth, worst
4	**Dijkstra's algorithm**	shortest, path, weighted, graph
5	**Bubble sort**	swap, adjacent, pass, compare
6	**Stack**	LIFO, push, pop, last
7	**In-order traversal**	left, node, right, ascending
8	**Heuristic (A*)**	estimate, guess, h(n), distance

Top Trumps — sorting & searching algorithms

How to play: Deal cards evenly, face down. Players hold their stack. The player to the left of the dealer picks a category (e.g. "Worst-case time") and reads its value. For **time complexity**, the **better (faster-growing-slower)** value wins — order from best to worst: O(1) < O(log n) < O(n) < O(n log n) < O(n^2). For **space**, lower wins. For "Needs sorted?", "No" beats "Yes" if you are racing to use it on raw data (house rule — agree before playing). Winner takes both cards. Tie = cards to the table, next round winner takes all.

All values verified against the OCR H446 spec.

```
+-----------------------------+   +-----------------------------+
| BUBBLE SORT                 |   | INSERTION SORT              |
| Best time:    O(n)*         |   | Best time:    O(n)          |
| Average time: O(n²)         |   | Average time: O(n²)         |
| Worst time:   O(n²)         |   | Worst time:   O(n²)         |
| Space:        O(1)          |   | Space:        O(1)          |
| Needs sorted? No            |   | Needs sorted? No            |
| Stable?       Yes           |   | Stable?       Yes           |
| * O(n) only with swap flag  |   |                             |
+-----------------------------+   +-----------------------------+

+-----------------------------+   +-----------------------------+
| MERGE SORT                  |   | QUICK SORT                  |
| Best time:    O(n log n)    |   | Best time:    O(n log n)    |
| Average time: O(n log n)    |   | Average time: O(n log n)    |
| Worst time:   O(n log n)    |   | Worst time:   O(n²)         |
| Space:        O(n)          |   | Space:        O(log n)      |
| Needs sorted? No            |   | Needs sorted? No            |
| Stable?       Yes           |   | Stable?       No            |
+-----------------------------+   +-----------------------------+

+-----------------------------+   +-----------------------------+
| LINEAR SEARCH               |   | BINARY SEARCH               |
| Best time:    O(1)          |   | Best time:    O(1)          |
| Average time: O(n)          |   | Average time: O(log n)      |
| Worst time:   O(n)          |   | Worst time:   O(log n)      |
| Space:        O(1)          |   | Space:        O(1) iter     |
| Needs sorted? No            |   | Needs sorted? YES           |
| Stable?       n/a           |   | Stable?       n/a           |
+-----------------------------+   +-----------------------------+
```

Quick-reference table (same data):

Card	Best	Average	Worst	Space	Needs sorted?	Stable?
Bubble sort	O(n)*	O(n²)	O(n²)	O(1)	No	Yes
Insertion sort	O(n)	O(n²)	O(n²)	O(1)	No	Yes
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	No	Yes
Quick sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No	No
Linear search	O(1)	O(n)	O(n)	O(1)	No	n/a
Binary search	O(1)	O(log n)	O(log n)	O(1) iter	Yes	n/a

* Bubble best case O(n) only with a swap flag on already-sorted data.

Quick-fire 'Name the Big O / algorithm'

How to play: One reader, rest of class buzz in. Reader reads the clue; first correct answer scores a point. All 12 answers verified below.

#	Clue	Answer
1	The time complexity of binary search (average/worst).	**O(log n)**
2	The time complexity of a single loop over n items.	**O(n)**
3	The worst-case time complexity of quick sort.	**$O(n^2)$**
4	The guaranteed time complexity of merge sort in all cases.	**O(n log n)**
5	The time complexity of pushing onto a stack.	**O(1)**
6	The search algorithm that needs the data sorted first.	**Binary search**
7	The sort that picks a pivot and partitions the list.	**Quick sort**
8	The traversal: Left → Node → Right.	**In-order**
9	The path-finding algorithm that adds a heuristic h(n) to g(n).	**A* search**
10	The data structure that is FIFO.	**Queue**
11	The sort that bubbles the largest value to the end each pass.	**Bubble sort**
12	The space complexity of merge sort.	**O(n)**

Self-check answers summary

- Big O order (best→worst growth): $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.
- Merge sort = always $O(n \log n)$ but $O(n)$ space; quick sort = $O(n \log n)$ average but $O(n^2)$ worst, $O(\log n)$ space.
- Binary search = $O(\log n)$, REQUIRES sorted data. Linear search = $O(n)$, works on unsorted.
- In-order traversal of a BST gives ascending order. DFS uses a stack; BFS uses a queue.
- Dijkstra = no heuristic, shortest path to all nodes; A* = f(n) = g(n) + h(n), heuristic-directed to one goal.